

锁存器，触发器，寄存器

以下内容包括059-storing-a-bit至062-registers的课件，如有错误请指正。

1 锁存器 latch

1.1 $\overline{R}\text{-}\overline{S}$ 锁存器 $\overline{R}\text{-}\overline{S}$ latch

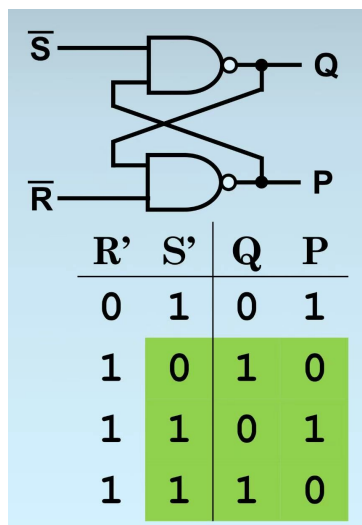


图 1: $\overline{R}\text{-}\overline{S}$ 锁存器的电路图和真值表

正如它的名字， $\overline{R}\text{-}\overline{S}$ 锁存器可以实现两个功能：储存数据和更新数据。首先要明确的是，对于两个输出Q和P，它们的值始终是互补的，这就意味着输出只有两种状态，一种是Q=1, P=0（我们称为置位 set），一种是Q=0, P=1（我们称为复位 reset）。当两个输入 $\overline{R}$ 和 $\overline{S}$ 都为1时，假设输出处于置位状态，那么 $\overline{R}$ 与QNAND的结果正好为0=P,同时P与 $\overline{S}$ NAND的结

果正好为 $1=Q$ ，这就说明这种状态是稳定的。同理，假设输出此时处于复位状态，我们同样可以推出稳定态的结果。这告诉我们当输入均为1时，无论输出处于置位还是复位状态，都能维持原有状态不变。

现在我们换一种情况，在输出处于置位状态时，假设 $\bar{R}$ 和 $\bar{S}$ 从11变为了 $\bar{S}$ 为1， $\bar{R}$ 为0，那么Q与 $\bar{R}$ NAND（1NAND0）的结果变为1，即新的P变成了1，随后这个新的P再与未发生改变的 $\bar{S}$ NAND（1NAND1）得到新的Q为0，同时这个新的Q与 $\bar{R}$ 再次NAND（0NAND0）会使P保持高电平的状态，最终使输出从原本的置位变为稳定的复位状态。

同样是输出处于置位，如果 $\bar{R}$ 不变， $\bar{S}$ 变为0，会发生什么呢？我们发现即使 $\bar{S}$ 变成了0，它与PNAND（0NAND0）的结果依然是1，说明Q等于1是不会变的，也就意味着置位的状态不会发生改变。

用同样的道理分析，如果输出处于复位，那么当输入均为1或 $\bar{R}$ 位0， $\bar{S}$ 为1时，它将保持原状态；当 $\bar{R}$ 为1， $\bar{S}$ 为0时，它将变为置位状态。

我们已经讨论了输入为11，10，01的情况，那假如输入为00，又会发生什么呢？事实上这种情况时无效的，原因是假如两个输入均为0，那么Q与P就会强制变为1，这与Q和P始终互补的基本原则是相矛盾的。

综上，我们可以得到如下表格，注意表格中的 $\bar{Q}$ 即为P。

表 7.1 低电平有效输入 $\bar{S}$ - $\bar{R}$ 锁存器的真值表

输 入		输 出		说 明
$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$	
1	1	NC	NC	没有变化，锁存器保持当前的状态
0	1	1	0	锁存器置位
1	0	0	1	锁存器复位
0	0	1	1	无效的情况

图 2: $\bar{R}$ - $\bar{S}$ 锁存器不同状态下实现的不同功能

1.2 门控D锁存器 Gated-D-latch

与 $\overline{R}\text{-}\overline{S}$ 锁存器不同的是，门控D锁存器的输入是D和WE(即该装置的门控部分)，输入经过处理后会传递给 $\overline{R}$ 和 $\overline{S}$ ，进入装置的锁存部分进行后续处理。其实现的功能是当WE为1时，D的值会传递给Q，WE为0时Q则将维持原值。电路图与真值表如下。

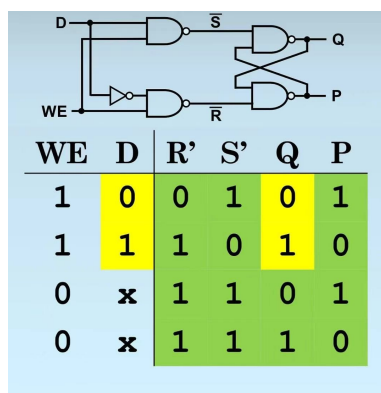


图 3: $\overline{R}\text{-}\overline{S}$ D门控锁存器的电路图和真值表

当WE等于0时，无论D为多少， $\overline{S}$ 和 $\overline{R}$ 都为1（因为无论0还是1和0NAND都为1），根据锁存器的原理，Q和P将会维持原值。当WE等于1时， $\overline{S}$ 会变成 $\overline{D}$ ，又因为 $Q=S$ ，所以Q会被赋以D的值。

2 触发器 flip-flop

2.1 D触发器 D flip-flop

与锁存器一样，触发器同样可以实现维持原有数据和更新数据的功能。但两者的区别在于锁存器是电平敏感，触发器是边沿敏感。这是什么意思呢？拿我们学过的门控D锁存器与D触发器为例，前者只要WE处于高电平状态（即为1）就能将D的数据更新至Q，而D触发器则必须在时钟处于上升沿的那一刹那才能将D的数据更新至Q。那么我们怎么在原有的锁存器的基础上做一些改动使其只能在时钟的上或下边沿更新数据呢？

方法有很多，一种是将门控D锁存器原有的WE输入改为clock并加入一个脉冲转换检测器，利用inverter的延迟生成一段时间很短的高电平，另一种是课件所采用的，利用两个clock相反的锁存器。当clock处于低电平

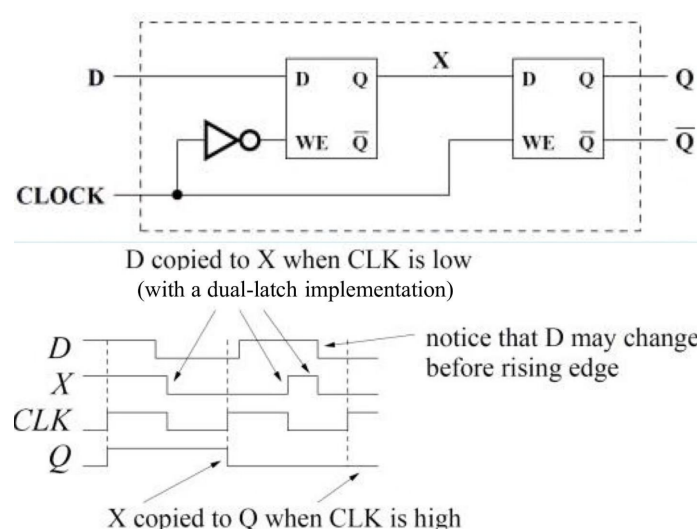


图 4: D触发器的电路图与波形图

时，第一个锁存器会打开，D的值会传递给X，随后在clock从低电平变成高电平的一瞬间，第二个锁存器被打开，之前储存在X的值又被传递给了输出Q。而接下来即使clock依旧维持在高电平，由于第一个锁存器是关闭的，因此新的D的值无法传递给X,也就无法传递给Q，必须等待下一个上升沿才行。（从波形图中我们也可以看出只有clk的上升沿处Q的值才会发生改变）如此我们就实现了从电平敏感到上升沿敏感的转变。

3 寄存器 register

寄存器是由许多个触发器组成的可以同时储存多位数据的元器件。除此之外，寄存器还能通过一些调整实现触发器所不能实现的功能。例如我想用引入一个独立于clock之外的开关来控制数据是维持原值还是更新数值，就可以通过在触发器的基础上增加一个选择器来实现。

如图5，LOAD作为这个2-1MUX的select bit,当它为1时IN的数值会输入至D中，当它为0时Q的值会被输入至D中。这样子处理的好处在于没有干扰CLK的运作，即不管怎样Q的值只会在CLK的上升沿发生变化，符合一般规律。

如图六，我们把这样子的一个1-bit register作为一个bit slice，将许多个bit slice并联在一起就可以得到并行输入的寄存器。

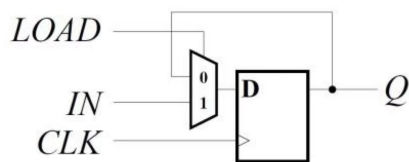


图 5: 一种1-bit register的电路图

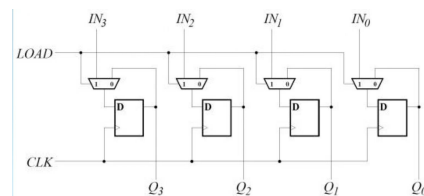


图 6: 并行输入的register

除了图六以外, 还有另一种由多个触发器组成的寄存器, 我们称之移位寄存器 (shift register), 其与前者相似的点是输出都为并行输出, 但区别在于前者是并行输入, 而后者是串行输入。每个clk周期中的上升沿, 单个

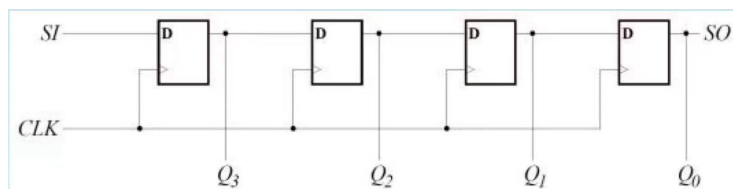


图 7: 移位寄存器的电路图

触发器都会录入上一个触发器的输出, 并将自己的输出传递给下一个触发器。即单个数据从输入进移位寄存器后每个clock周期都向右移一位。

通过其能将串行输入转换为并行输出 (串并转换, deserialization) 的特点, 我们可以实现许多功能。例如从外部输入进了速度为100Gbps (即一秒钟传递 100×10^{10} 个比特) 的数据, 而计算机内部电路的clock频率只有4GHz, 其处理数据的速度跟不上外界输入数据的速度, 该怎么办呢? 我们

可以采用一个25-bit的移位寄存器，由于移位寄存器的输入是相对而言逻辑简单的串行输入，其clock速率是可以跟上数据输入的100GHz的速率的，即每周期10ps。也就是每经过10ps移位寄存器都会把数据向右移动一位，这样经过25个周期，250ps后，移位寄存器的25个触发器就都装有了数据，此时我们将这25位数据并行提取出来进入下一步处理。从整体来看，我们就是每250ps为一个周期提取了一次数据，对应的频率恰好是4GHz,即计算机内部clock的频率。如此我们就让从外界提取数据的速度等于了计算机内部处理数据的速度。

当然，我们可以将移位寄存器的每个bit slice换成如图五一样的设计，让每个bit slice公用一个LOAD，这样就可以通过控制LOAD随时控制移位寄存器是否要把数据右移一位。

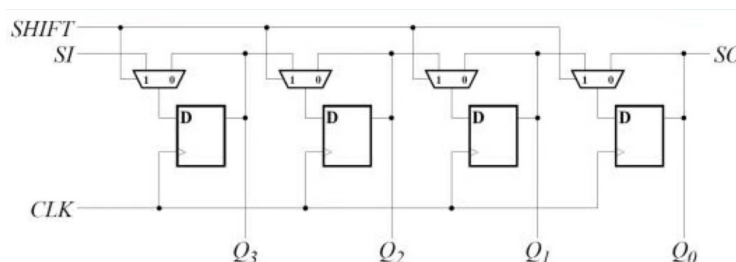


图 8: 可控移位寄存器的电路图